

Tiền xử lý Dữ liệu với Python

Làm sạch, biến đổi và chuẩn hóa dữ liệu cho Phân tích & Machine Learning

DSAI1005 - Phân tích Dữ liệu với Python

Khoa Khoa học Dữ liệu và Trí tuệ Nhân tạo
Đại học Kinh tế Quốc dân

2026

Lộ trình bài học

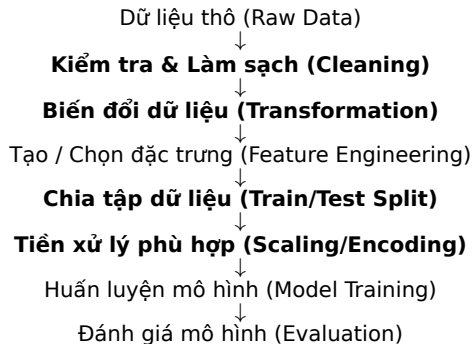
- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Động lực của bài học

Trong thực tế, dữ liệu thu thập được hiếm khi sẵn sàng để đưa ngay vào phân tích hay mô hình học máy:

- **Giá trị bị thiếu (Missing values):** do người dùng bỏ trống hoặc lỗi truyền nhận.
- **Giá trị bất thường (Outliers):** có thể là lỗi nhập liệu hoặc khách hàng đặc biệt.
- **Định dạng chưa đúng:** số được lưu thành chuỗi, ngày tháng sai định dạng.
- **Biến phân loại (Categorical variables):** dạng chuỗi cần chuyển thành dạng số.
- **Thang đo chênh lệch (Scale difference):** thu nhập hàng chục triệu vs. tuổi dưới 100.
- **Mất cân bằng lớp (Class imbalance):** tỷ lệ gian lận hoặc bệnh tật rất thấp.

Vị trí của Tiền xử lý trong Quy trình Học máy



Vai trò then chốt:

- Nâng cao chất lượng dữ liệu đầu vào.
- Giảm tác động tiêu cực của nhiễu.
- Giúp các thuật toán hoạt động chính xác.
- **Hạn chế Data Leakage:** Tránh rò rỉ thông tin từ tập test vào tập train.
- Giúp mô hình dễ diễn giải hơn.

Vì sao thuật toán cần Tiền xử lý?

Nhiều thuật toán học máy rất nhạy cảm với thang đo của biến:

- Giả sử có hai biến: Income (5.000.000 – 100.000.000) và Age (18 – 70).
- Khi dùng khoảng cách Euclidean:

$$d = \sqrt{(\text{Income}_1 - \text{Income}_2)^2 + (\text{Age}_1 - \text{Age}_2)^2}$$

- Biến Income với độ lớn vượt trội sẽ **chi phối gần như toàn bộ** khoảng cách, khiến biến Age mất hoàn toàn ý nghĩa!

Các thuật toán chịu ảnh hưởng mạnh:

- k-Nearest Neighbors (k-NN), k-Means Clustering
- Support Vector Machines (SVM)
- Hồi quy có điều chuẩn (Ridge, Lasso Logistic/Linear Regression)
- Mạng nơ-ron nhân tạo (Neural Networks)

Mục tiêu học tập

Sau bài học này, người học có thể:

- 1 Sử dụng Pandas để kiểm tra cấu trúc, kiểu dữ liệu và giá trị thiếu.
- 2 Phân biệt **missing value**, **outlier** và giá trị không hợp lệ.
- 3 Sử dụng thống kê mô tả và biểu đồ (boxplot, histogram) để đánh giá dữ liệu.
- 4 Áp dụng phương pháp IQR để phát hiện ngoại lệ và biết khi nào nên xử lý.
- 5 Xử lý các biến phân loại bằng kỹ thuật **One-Hot Encoding**.
- 6 Phân biệt và áp dụng đúng **Normalization** (Min-Max) và **Standardization** (Z-score).
- 7 Hiểu nguyên tắc **chống Data Leakage**: Các phép biến đổi phải được học từ **Training set**.
- 8 Xây dựng quy trình tiền xử lý hoàn chỉnh bằng **ColumnTransformer** và **Pipeline** của scikit-learn.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu**
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Bước 1: Import thư viện và nạp dữ liệu

Bộ dữ liệu minh họa: `diabetes.csv` (Bộ dữ liệu chẩn đoán bệnh tiểu đường).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.impute import SimpleImputer

# Đọc dữ liệu
df = pd.read_csv("diabetes.csv")
df.head()
```

`df.head()` giúp quan sát 5 dòng đầu tiên: nhận diện tên cột, kiểu dữ liệu sơ bộ và các điểm bất thường dễ thấy.

Kết quả nạp dữ liệu ban đầu

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	
0	6	148	72	35	0	33.6	0.627	50	1	
1	1	85	66	29	0	26.6	0.351	31	0	
2	8	183	64	0	0	23.3	0.672	32	1	
3	1	89	66	23	94	28.1	0.167	21	0	
4	0	137	40	35	168	43.1	2.288	33	1	

Các lệnh kiểm tra nhanh cấu trúc

Kích thước và danh sách cột:

```
# So dòng và số cột  
df.shape  
  
# Tra về: (768, 9)  
  
# Danh sách tên cột  
df.columns  
  
# Kiểm tra tổng số phần tử  
df.size
```

Quan sát các mẫu dữ liệu:

```
# 10 dòng đầu tiên  
df.head(10)  
  
# 5 dòng cuối cùng  
df.tail()  
  
# 5 dòng ngẫu nhiên  
df.sample(5)
```

Lưu ý

Quan sát `df.sample(5)` thường khách quan hơn chỉ xem `head()` hoặc `tail()` vì dữ liệu thực tế có thể đã bị sắp xếp theo một thứ tự cụ thể.

Bước 2: Kiểm tra kiểu dữ liệu với `df.info()`

`df.info()` cung cấp tổng quan về: số lượng dòng, số cột, tên cột, số giá trị non-null, kiểu dữ liệu (dtype) và dung lượng bộ nhớ.

```
df.info()
```

Kiểu dữ liệu phổ biến	Ý nghĩa
-----------------------	---------

int64	Số nguyên (tuổi, số lượng đơn hàng, đếm)
float64	Số thực (doanh thu, nồng độ, trọng lượng, tỷ lệ)
object	Thường là chuỗi văn bản hoặc dữ liệu hỗn hợp
bool	Giá trị logic True/False
datetime64	Thời gian (ngày, giờ giao dịch)
category	Biến phân loại có số lượng nhóm hữu hạn

Kiểm tra giá trị thiếu (Missing Values)

Kiểm tra số lượng và tỷ lệ giá trị thiếu theo từng cột:

```
# Dem so gia tri thieu theo tung cot
df.isna().sum() # Hoac df.isnull().sum()

# Tinh ty le phan tram gia tri thieu
missing_rate = df.isna().mean() * 100
print(missing_rate)
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0
dtype: int64	

Cảnh báo quan trọng: Giá trị thiếu tiềm ẩn

Missing value không phải lúc nào cũng là NaN!

Trong thực tế, nhiều hệ thống ghi nhận giá trị thiếu bằng các mã quy ước:

0, -1, 999, "Unknown", "N/A", "?"

Ví dụ trong tập dữ liệu Y tế (diabetes.csv):

- Các biến sinh học như Glucose, BloodPressure, SkinThickness, Insulin, BMI có giá trị bằng **0**.
- Về mặt y khoa, một người sống không thể có huyết áp hay đường huyết bằng 0!
- Đây thực chất là **dữ liệu bị thiếu** nhưng được lưu dưới dạng số 0.

Ví dụ trong Kinh doanh:

- Age = 0 hoặc Age = -1; Delivery_Time = 999 ngày; Revenue = 0.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ**
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Bước 3: Thống kê mô tả với `df.describe()`

```
# Thống kê mô tả cho các biến số  
df.describe()  
  
# Thống kê mô tả cho toàn bộ các biến (bao gồm cả categorical)  
df.describe(include="all")
```

Các chỉ số then chốt cần quan sát:

- **min và max:** Có nằm ngoài khoảng giá trị hợp lý không? (Ví dụ: Age có $\text{min} = -3$ hoặc $\text{max} = 245$).
- **mean (trung bình) vs. 50% (trung vị - median):**
 - Nếu $\text{mean} \approx \text{median}$: phân phối tương đối đối xứng.
 - Nếu $\text{mean} \gg \text{median}$: phân phối lệch phải (right-skewed), có sự hiện diện của các giá trị cực lớn kéo trung bình lên.

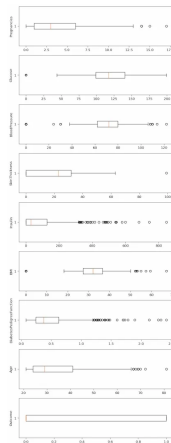
Bước 4: Trực quan hóa ngoại lệ bằng Boxplot

Boxplot là công cụ hàng đầu để nhận diện hình dạng phân phối và các điểm dị biệt:

```
# Vẽ boxplot cho 1 biến
plt.figure(figsize=(8, 3))
plt.boxplot(df["Insulin"].dropna(), vert=False)
plt.xlabel("Insulin")
plt.title("Boxplot of Insulin")
plt.show()

# Vẽ boxplot cho tất cả các biến số trên subplots
numeric_cols = df.select_dtypes(include=np.number).columns
fig, axes = plt.subplots(len(numeric_cols), 1, figsize=(8, 2.5 * len(numeric_cols)))
for i, col in enumerate(numeric_cols):
    axes[i].boxplot(df[col].dropna(), vert=False)
    axes[i].set_title(col)
plt.tight_layout()
```


Biểu đồ Boxplot của các biến số



Biểu đồ Histogram & Đánh giá Phân phối

Histogram kết hợp đường cong mật độ (KDE) giúp quan sát cụ thể độ lệch:

```
# Ve bang Pandas
df["Insulin"].hist(bins=30)
plt.xlabel("Insulin")
plt.ylabel("Tan suat")
plt.show()

# Ve bang Seaborn co duong KDE
sns.histplot(data=df, x="Insulin", kde=True)
plt.title("Phan phoi bien Insulin")
plt.show()
```

Nhận xét

Nếu phân phối bị lệch nặng (heavy-tailed / skewed), việc áp dụng phép biến đổi logarit hoặc chuẩn hóa bằng trung vị (median) thường hiệu quả hơn trung bình (mean).

Ngoại lệ (Outlier) là gì?

Outlier là quan sát nằm xa phần lớn các điểm dữ liệu còn lại trong tập dữ liệu.

Lưu ý cốt lõi

Outlier KHÔNG ĐỒNG NGHĨA với dữ liệu sai!

Ví dụ trong thực tế:

- Một khách hàng VIP chi tiêu 500 triệu đồng trong khi đa số khách hàng chỉ chi tiêu 2 triệu đồng.
- Một đơn hàng vận chuyển quốc tế mất 45 ngày so với trung bình 3 ngày.
- Trong **Phát hiện gian lận (Fraud Detection)**, các giao dịch bất thường chính là mục tiêu quan trọng nhất cần tìm kiếm!

Quy tắc: Mục tiêu đầu tiên luôn là **phát hiện và điều tra nguyên nhân**, tuyệt đối không tự động xóa bỏ mà không hiểu ngữ cảnh bài toán.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR**
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Công thức Interquartile Range (IQR)

Phương pháp IQR dựa trên tứ phân vị, không bị ảnh hưởng bởi các giá trị cực trị:

- Q_1 (Phân vị 25%): 25% dữ liệu nhỏ hơn giá trị này.
- Q_3 (Phân vị 75%): 75% dữ liệu nhỏ hơn giá trị này.

Độ trải giữa (IQR) và Ngưỡng xác định

$$IQR = Q_3 - Q_1$$

$$\text{Ngưỡng dưới (Lower Bound)} = Q_1 - 1.5 \times IQR$$

$$\text{Ngưỡng trên (Upper Bound)} = Q_3 + 1.5 \times IQR$$

Các giá trị nằm ngoài đoạn [Lower, Upper] được coi là điểm ngoại lệ tiềm năng.

Cài đặt phương pháp IQR trong Python

```
# Tính Q1, Q3 và IQR
q1, q3 = np.percentile(df["Insulin"].dropna(), [25, 75])
iqr = q3 - q1

lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

print(f"Ngưỡng dưới: {lower:.2f} | Ngưỡng trên: {upper:.2f}")

# Lọc ra các quan sát ngoại lệ
outliers = df[(df["Insulin"] < lower) | (df["Insulin"] > upper)]
print("Số lượng ngoại lệ:", len(outliers))

# Nếu có lý do hợp lý để loại bỏ:
clean_df = df[(df["Insulin"] >= lower) & (df["Insulin"] <= upper)].copy()
```

5 Câu hỏi trước khi xử lý Ngoại lệ

Trước khi quyết định xử lý một điểm ngoại lệ, người phân tích cần tự hỏi:

- ❶ Giá trị này có **bất khả thi về mặt nghiệp vụ** không? (Ví dụ: Tuổi = -5 hay 200 tuổi).
- ❷ Có phải là **lỗi nhập liệu** hoặc lỗi hệ thống ghi nhận không? (Ví dụ: gõ thêm số 0).
- ❸ Có sự **sai lệch về đơn vị đo** không? (Ví dụ: gram vs. kilogram, VNĐ vs. USD).
- ❹ Ngoại lệ có phải chính là **đối tượng mục tiêu** cần phát hiện không? (Gian lận thẻ tín dụng, lỗi động cơ máy bay).
- ❺ Việc xóa có làm **mất đi một phân khúc quan trọng** của bài toán không?

Các phương án xử lý ngoài việc xóa bỏ:

- Giữ nguyên và dùng mô hình ít nhạy với outlier (Decision Tree, Random Forest).
- Cắt ngọn (Winsorization / Capping) đưa về ngưỡng Lower/Upper.
- Biến đổi dữ liệu (Log transform, Box-Cox) để thu hẹp độ phân tán.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)**
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Các chiến lược xử lý Missing Data cơ bản

1. Loại bỏ dòng hoặc cột:

```
# Xóa tất cả dòng có giá trị thiếu
df_clean = df.dropna()

# Xóa các dòng thiếu ở cột cụ thể
df_clean = df.dropna(subset=["Age"])

# Xóa cột nếu thiếu quá nhiều (>60%)
df_clean = df.drop(columns=["Col"])
```

Chỉ nên drop khi tỷ lệ thiếu rất nhỏ và không làm sai lệch phân phối.

2. Điền giá trị thống kê (Imputation):

```
# Điền bằng Trung bình (Mean)
df["Age"] = df["Age"].fillna(
    df["Age"].mean()
)

# Điền bằng Trung vị (Median) - tốt khi có outlier
df["Income"] = df["Income"].fillna(
    df["Income"].median()
)

# Điền biến phân loại bằng Mode
df["City"] = df["City"].fillna(
    df["City"].mode()[0]
)
```

SimpleImputer trong Scikit-Learn

Scikit-Learn cung cấp lớp SimpleImputer chuẩn hóa việc điền dữ liệu:

```
from sklearn.impute import SimpleImputer

# Chien luoc dien trung vi cho bien so
imputer_num = SimpleImputer(strategy="median")

# Chien luoc dien yeu vi cho bien phan loai
imputer_cat = SimpleImputer(strategy="most_frequent")

# Chien luoc dien hang so
imputer_const = SimpleImputer(strategy="constant", fill_value="Unknown")
```

Các tham số chính:

- strategy: "mean", "median", "most_frequent", "constant".
- fill_value: Giá trị cần điền khi dùng chiến lược "constant".

Cảnh báo quan trọng: Data Leakage khi Imputation

Quy tắc phòng chống rò rỉ dữ liệu (Data Leakage)

KHÔNG BAO GIỜ tính mean hay median trên toàn bộ tập dữ liệu rồi điền cho cả tập train và test!

Dữ liệu Train $\rightarrow$ Học tham số (median) $\rightarrow$ Điền cho cả Train & Test

Lý do then chốt:

- Tập test đóng vai trò là dữ liệu mới trong tương lai.
- Nếu sử dụng thông tin của tập test để tính toán tham số tiền xử lý, mô hình sẽ đạt kết quả đánh giá quá lạc quan nhưng hoạt động kém khi triển khai thực tế.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)**
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Vì sao cần mã hóa biến phân loại?

Hầu hết các thuật toán học máy (Linear/Logistic Regression, Neural Networks, SVM) đều tính toán trên ma trận số. Do đó, các biến phân loại:

$$\text{Region} \in \{\text{"North"}, \text{"Central"}, \text{"South"}\}$$

cần được chuyển hóa thành các cột nhị phân (0 và 1).

Cách 1: One-Hot Encoding bằng Pandas (pd.get_dummies)

```
# drop_first=True để tránh đa cộng tuyến (dummy variable trap)
df_encoded = pd.get_dummies(df, columns=["Region"], drop_first=True)
```

Region (Gốc)	Region_Central	Region_North	Region_South (Bị bỏ nếu drop_first)
Central	1	0	0
North	0	1	0
South	0	0	1

OneHotEncoder trong Scikit-Learn

Trong quy trình học máy chuyên nghiệp, nên dùng OneHotEncoder của scikit-learn:

```
from sklearn.preprocessing import OneHotEncoder

# handle_unknown='ignore' rất quan trọng khi gặp nhãn mới o tập Test
encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

# Học các categories từ tập train và biến đổi
X_train_encoded = encoder.fit_transform(X_train[["Region"]])
X_test_encoded = encoder.transform(X_test[["Region"]])
```

Tại sao `handle_unknown="ignore"` lại hữu ích?

Nếu tập Test xuất hiện một giá trị mới (ví dụ: "Highlands") chưa từng có trong tập Train, `pd.get_dummies` sẽ tạo số lượng cột khác nhau hoặc gây lỗi mô hình, còn `handle_unknown="ignore"` sẽ gán toàn bộ số 0 cho các cột tương ứng một cách an toàn.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu**
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Bước 8: Phân tích Tương quan (Correlation Analysis)

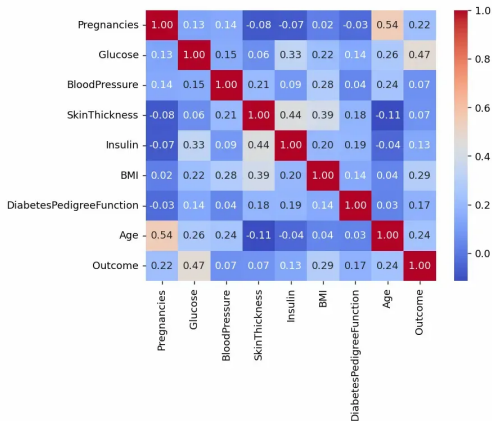
Hệ số tương quan tuyến tính Pearson $r \in [-1, 1]$:

```
# Tính ma trận tương quan cho các biến số
corr = df.corr(numeric_only=True)

# Trực quan hóa bằng Heatmap
plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()

# Kiểm tra tương quan với biến mục tiêu Outcome
corr["Outcome"].sort_values(ascending=False)
```


Kết quả Tương quan với Biến mục tiêu



Nhận xét từ kết quả:

- Glucose ($r = 0.47$) và BMI ($r = 0.29$) có tương quan thuận mạnh nhất với bệnh tiểu đường (Outcome).
- Age ($r = 0.24$) cũng có mối quan hệ tích cực.
- **Cảnh báo:** Tương quan cao giữa 2 biến đầu vào có thể gây **đa cộng tuyến**.
- Tương quan thấp không đồng nghĩa biến vô dụng vì quan hệ có thể là phi tuyến.

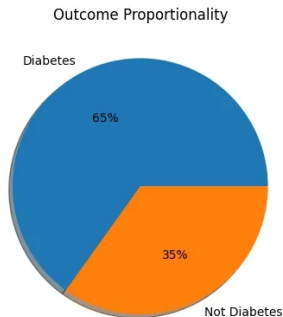
Bước 9: Kiểm tra Phân phối Biến mục tiêu (Target Distribution)

Đối với bài toán phân loại, kiểm tra tỷ lệ giữa các lớp là bắt buộc:

```
# Dem so luong va ty le
print(df["Outcome"].value_counts())
print(df["Outcome"].value_counts(normalize=True))

# Truc quan hoa bang countplot hoac pie chart
counts = df["Outcome"].value_counts()
plt.pie(counts.values, labels=counts.index, autopct="%.1f%%")
plt.title("Outcome Distribution")
plt.show()
```

Phân phối Lớp trong diabetes.csv



- Lớp 0 (Không mắc bệnh): 65.1% | Lớp 1 (Mắc bệnh): 34.9%.
- Dữ liệu có sự mất cân bằng nhẹ. Nếu dữ liệu mất cân bằng nghiêm trọng (ví dụ 99% vs 1%), chỉ số Accuracy sẽ mất ý nghĩa, cần dùng **Precision, Recall, F1-score, ROC-AUC** và chia tập theo **Stratified Sampling**.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test**
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Bước 10: Tách Biến đầu vào (X) và Biến mục tiêu (y)

```
# X là ma trận các đặc trưng (features)
X = df.drop(columns=["Outcome"])

# y là vector biến mục tiêu cần dự đoán (target)
y = df["Outcome"]

print("Kích thước X:", X.shape)
print("Kích thước y:", y.shape)
```

Ý nghĩa:

- X đại diện cho thông tin quan sát được tại thời điểm đưa ra quyết định.
- y là nhãn cần dự báo.
- Đảm bảo trong X không chứa thông tin của y hoặc các biến chỉ xuất hiện sau khi y đã xảy ra (**Target Leakage**).

Bước 11: Chia Training Set và Test Set

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% giữ lại làm tập test
    random_state=42, # Có định seed để tái lập kết quả
    stratify=y # Giữ nguyên tỷ lệ các lớp giữa train và test
)
```

Giải thích các tham số quan trọng:

- `test_size=0.2`: 80% dữ liệu dùng để học, 20% dùng để kiểm định khách quan.
- `random_state=42`: Đảm bảo mỗi lần chạy đều phân chia giống nhau.
- `stratify=y`: Đảm bảo tỷ lệ người mắc bệnh trong cả tập Train và Test đều giữ nguyên ở mức $\approx 35\%$.

Nguyên tắc Vàng: Vì sao phải chia trước khi Scaling?

SAI LẦM (Gây Data Leakage)

```
scaler = StandardScaler()
# Scaler học trên TOÀN BỘ dữ liệu
X_scaled = scaler.fit_transform(X)

# Sau đó mới chia train/test
X_tr, X_te = train_test_split(X_scaled)
```

→ Giá trị trung bình μ và độ lệch chuẩn σ của tập Test đã bị rò rỉ vào quá trình học!

CÁCH LÀM ĐÚNG

```
# Chia train/test trước
X_tr, X_te, y_tr, y_te = (
    train_test_split(X, y, test_size=0.2)
)

scaler = StandardScaler()
# CHI fit trên Train
X_tr_scaled = scaler.fit_transform(X_tr)
# Test CHI được transform
X_te_scaled = scaler.transform(X_te)
```

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)**
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Hai kỹ thuật chuẩn hóa đặc trưng cốt lõi

1. Min-Max Scaling

(Normalization) Đưa dữ liệu về đoạn cố định, thường là $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- Giá trị nhỏ nhất biến thành 0.
- Giá trị lớn nhất biến thành 1.
- Rất nhạy cảm với các điểm ngoại lệ (vì $x_{\min}$ và $x_{\max}$ bị kéo giãn).

2. Standardization (Z-score) Biến đổi dữ liệu có trung bình 0 và độ lệch chuẩn 1:

$$z = \frac{x - \mu}{\sigma}$$

- Không giới hạn miền giá trị cố định.
- Ít bị chi phối bởi outlier hơn Min-Max.
- Không biến phân phối thành phân phối chuẩn nếu gốc không chuẩn!

Thực hành Min-Max Scaling & Kết quả

```
from sklearn.preprocessing import MinMaxScaler
```

```
minmax_scaler = MinMaxScaler()
X_train_norm = minmax_scaler.fit_transform(X_train)
X_test_norm = minmax_scaler.transform(X_test)
```

```
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.        ,
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.        ,
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.        , 0.        ,
        0.34724292, 0.25362938, 0.18333333],
       [0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
        0.41877794, 0.03800171, 0.        ],
       [0.        , 0.68844221, 0.32786885, 0.35353535, 0.19858156,
        0.64232489, 0.94363792, 0.2        ]])
```

Thực hành Standardization & Kết quả

```
from sklearn.preprocessing import StandardScaler
```

```
standard_scaler = StandardScaler()
X_train_std = standard_scaler.fit_transform(X_train)
X_test_std = standard_scaler.transform(X_test)
```

```
array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,
         0.20401277,  0.46849198,  1.4259954 ],
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -0.69289057,
        -0.68442195, -0.36506078, -0.19067191],
       [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -0.69289057,
        -1.10325546,  0.60439732, -0.10558415],
       [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  0.12330164,
        -0.49404308, -0.92076261, -1.04154944],
       [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  0.76583594,
         1.4097456 ,  5.4849091 , -0.0204964 ]])
```

So sánh Min-Max Scaling và Standardization

Tiêu chí	Min-Max Scaling	Standardization
Công thức	$\frac{X - X_{\min}}{X_{\max} - X_{\min}}$	$\frac{X - \mu}{\sigma}$
Phạm vi giá trị	Thường là $[0, 1]$	Không giới hạn cố định
Trung bình	Không nhất thiết bằng 0	Bằng 0 trên tập Train
Độ lệch chuẩn	Thay đổi theo dữ liệu	Bằng 1 trên tập Train
Độ nhạy Outlier	Rất cao	Trung bình
Ứng dụng tiêu biểu	Thuật toán cần khoảng bounded, ảnh (0-255), k-NN	Ridge/Lasso, SVM, PCA, Neural Networks

Mô hình nào cần Feature Scaling?

CẦN Feature Scaling

Các mô hình tính khoảng cách hoặc dùng đạo hàm để cập nhật trọng số:

- k-Nearest Neighbors (k-NN)
- k-Means Clustering
- Support Vector Machines (SVM)
- Principal Component Analysis (PCA)
- Hồi quy tuyến tính có Regularization
- Mạng Nơ-ron (Deep Learning)

KHÔNG CẦN Feature Scaling

Các mô hình dựa trên Cây quyết định:

- Decision Trees
- Random Forests
- Gradient Boosted Trees (XGBoost, LightGBM, CatBoost)

Lý do: Cây phân nhánh dựa trên thứ tự giá trị của từng đặc trưng độc lập, hoàn toàn không bị ảnh hưởng bởi độ lớn thang đo.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp**
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Xử lý Dữ liệu Hỗn hợp với ColumnTransformer

Dữ liệu thực tế thường gồm cả biến số và biến phân loại:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Pipeline cho biến số: Diện median -> Chuẩn hóa Z-score
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Pipeline cho biến phân loại: Diện mode -> One-Hot Encoding
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Kết hợp bằng ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, ["Age", "Income", "Order_Value"]),
    ("cat", categorical_transformer, ["Region", "Customer_Type"])
])
```

Tích hợp toàn bộ vào Pipeline Học máy

Đóng gói toàn bộ Tiền xử lý và Mô hình dự báo:

```
from sklearn.linear_model import LogisticRegression

# Đóng gói preprocessor và model vào Pipeline chung
full_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

# Huan luyen: Scikit-learn tự động fit trên train, không leak dữ liệu!
full_pipeline.fit(X_train, y_train)

# Dự đoán trên tập test
y_pred = full_pipeline.predict(X_test)
```

Lợi ích vượt trội của Pipeline:

- ❶ Loại bỏ hoàn toàn nguy cơ rò rỉ dữ liệu (Data Leakage).
- ❷ Mã nguồn ngắn gọn, tường minh và dễ bảo trì.
- ❸ Triển khai dự báo trên dữ liệu mới (production) an toàn tuyệt đối.

Lộ trình bài học

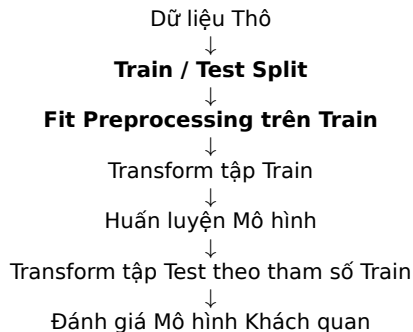
- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến**
- 12 Bài tập Thực hành & Tự kiểm tra

Quy trình Tiền xử lý Dữ liệu Chuẩn mực

Trình tự 10 bước an toàn:

- 1 **Understand:** Hiểu bài toán và ngữ cảnh.
- 2 **Inspect:** Kiểm tra shape, info, dtypes.
- 3 **Explore (EDA):** Thống kê mô tả, boxplot, heatmap.
- 4 **Split:** Chia Train/Validation/Test.
- 5 **Handle Missing:** Điền dữ liệu trên Train.
- 6 **Encode:** Mã hóa biến phân loại.
- 7 **Scale:** Chuẩn hóa thang đo biến số.
- 8 **Pipeline:** Đóng gói các bước

Sơ đồ luồng an toàn



Những lỗi phổ biến khi Tiền xử lý (Phần 1: Dữ liệu & Ngoại lệ)

❶ Tự động xóa mọi Outlier:

- Sai vì ngoại lệ có thể là quan sát hoàn toàn hợp lệ hoặc là mục tiêu chính cần phát hiện (ví dụ: giao dịch gian lận).

❷ Chỉ kiểm tra `isna()`:

- Bỏ sót các giá trị thiếu ngầm định (0, -1, 999, "?", "Unknown").

❸ Chuẩn hóa cả biến mục tiêu (y):

- Với bài toán phân loại, y là nhãn nhị phân (0/1), không được scale.

❹ Loại bỏ biến chỉ vì tương quan tuyến tính thấp:

- Tương quan Pearson chỉ đo mối quan hệ tuyến tính; biến có thể chứa thông tin phi tuyến rất quan trọng.

Những lỗi phổ biến khi Tiền xử lý (Phần 2: Leakage & Đánh giá)

5 Fit Scaler / Imputer trước khi chia Train/Test:

- Đây là dạng **Data Leakage** phổ biến nhất, khiến kết quả đánh giá mô hình bị sai lệch so với thực tế.

6 Chọn biến có tương quan quá cao với y :

- Nguy cơ **Target Leakage** (ví dụ biến `Sales_Tax` được tính trực tiếp từ doanh số cần dự báo `Total_Sales`).

7 Xử lý tập Train và tập Test bằng các quy tắc khác nhau:

- Gây sai lệch phân phối (distribution drift), mô hình dự đoán sai hoàn toàn khi triển khai trên dữ liệu mới.

Case Study: Dự đoán Khách hàng Rời bỏ (Churn)

Tập dữ liệu doanh nghiệp viễn thông:

- Customer_ID, Age, Monthly_Spending, Tenure_Months, Region, Support_Calls, Churn

Các bước tiền xử lý chuẩn mực:

- 1 Loại bỏ Customer_ID vì không có giá trị dự báo.
- 2 Kiểm tra Age = -1 → Thay thế bằng NaN rồi điền bằng trung vị.
- 3 Chuẩn hóa Region: "Hanoi", "Ha Noi", "HN" → quy về một nhãn "Hanoi".
- 4 Kiểm tra ngoại lệ của Support_Calls bằng boxplot; giữ lại vì khách hàng gọi hỗ trợ nhiều có nguy cơ rời bỏ cao.
- 5 Tách X, y và chia Train/Test theo phương pháp stratify=y.
- 6 Xây dựng Pipeline: Điền median & Scale biến số; One-hot encode biến phân loại.

Lộ trình bài học

- 1 Tổng quan về Tiền xử lý Dữ liệu
- 2 Bước 1 & 2 — Nạp và Kiểm tra Cấu trúc Dữ liệu
- 3 Bước 3 & 4 — Thống kê Mô tả và Phát hiện Ngoại lệ
- 4 Bước 5 — Phát hiện Ngoại lệ bằng Phương pháp IQR
- 5 Bước 6 — Xử lý Giá trị Thiếu (Missing Data)
- 6 Bước 7 — Xử lý Biến Phân loại (Categorical Encoding)
- 7 Bước 8 & 9 — Phân tích Tương quan và Biến Mục tiêu
- 8 Bước 10 & 11 — Tách Features/Target và Chia Train/Test
- 9 Bước 12 — Chuẩn hóa Đặc trưng (Feature Scaling)
- 10 Bước 13 — Xây dựng Pipeline Tiền xử lý Chuyên nghiệp
- 11 Quy trình Tổng quát & Các Bẫy Sai sót Phổ biến
- 12 Bài tập Thực hành & Tự kiểm tra

Bài tập Thực hành Ngắn

Bài 1 (Kiểm tra dữ liệu): Cho DataFrame `df`. Viết code tính tỷ lệ phần trăm missing values của từng cột và hiển thị theo thứ tự giảm dần.

Bài 2 (Missing Data): Viết code điền các giá trị thiếu trong cột `Income` bằng giá trị trung vị của cột đó được tính trên `X_train`.

Bài 3 (Xác định Outlier IQR):

```
# Điền vào cho trong để tính ngưỡng trên và ngưỡng dưới
q1 = df["Order_Value"].quantile(0.25)
q3 = df["Order_Value"].quantile(0.75)
iqr = _____
lower_bound = _____
upper_bound = _____
```

Câu hỏi Ôn tập & Trắc nghiệm

Câu 1. Khi một biến số có phân phối lệch phải rất nặng và nhiều ngoại lệ cực lớn, giá trị thống kê nào phù hợp hơn để diễn dữ liệu thiếu?

- A. Trung bình (Mean)
- B. Trung vị (Median)
- C. Giá trị nhỏ nhất (Minimum)
- D. Số 0

Câu 2. Với StandardScaler, cách áp dụng nào sau đây là **đúng chuẩn**?

- A. `scaler.fit_transform(X_train)` và `scaler.fit_transform(X_test)`
- B. `scaler.fit(X)` rồi `train_test_split(X)`
- C. `scaler.fit_transform(X_train)` và `scaler.transform(X_test)`
- D. `scaler.transform(X_train)` và `scaler.fit(X_test)`

Tóm tắt Nội dung Bài học

- **Tiền xử lý dữ liệu** là nền tảng quyết định chất lượng của mọi mô hình học máy.
- **Missing Values:** Luôn lưu ý các giá trị thiếu tiềm ẩn (0, -1, 999); dùng SimpleImputer.
- **Outliers:** Dùng Boxplot và phương pháp IQR để phát hiện; tìm hiểu nguyên nhân trước khi quyết định xử lý.
- **Categorical Encoding:** Sử dụng One-Hot Encoding với `handle_unknown="ignore"` để xử lý nhãn mới.
- **Scaling:** Phân biệt Min-Max Scaling (đoạn $[0, 1]$) và Standardization ($\mu = 0, \sigma = 1$).
- **Chống Data Leakage:** Luôn chia Train/Test trước; chỉ fit trên Train, dùng tham số đó để transform Test.
- **Pipeline & ColumnTransformer:** Công cụ chuẩn mực của scikit-learn giúp đóng gói toàn bộ quy trình tiền xử lý an toàn và chuyên nghiệp.

Tài liệu Tham khảo

- 1 Scikit-Learn Documentation. *Dataset transformations: preprocessing, imputation, pipelines*.
<https://scikit-learn.org/stable/modules/preprocessing.html>
- 2 Shmueli, G., Bruce, P. C., Deokar, A. V., & Patel, N. R. (2023). *Machine Learning for Business Analytics: Concepts, Techniques and Applications in RapidMiner*. Wiley (Chương 2).
- 3 Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- 4 McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.